Bank System Analysis

Ahmed EL Sheikh
ZLO1PI

Course name: Basics of Programming 2

Course ID: BMEVIIIAA03

Name: Ahmed Yasser Ali Refaat El Sheikh

Neptune code: ZLO1PI

Lab group ID: CS16D

Instructor name: Vatikus Marton

Project title: Bank System

Type of project: Final Project

Submission date: 13 / 5 / 2025

Introduction:

This project is a C++ based bank management system simulation that enables users to create, manage, and interact with different types of accounts (Standard, Savings, and Business).

It allows secure transactions, personal information updates, and stores data in files after exiting the code.

This code

Program interface:

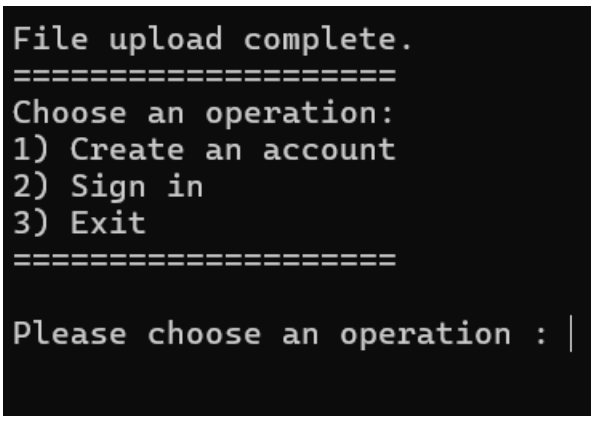
Compiling the program through the file [Bank_System.cpp], using any sort of C++ compiler.

The user interacts with the program through the terminal. Preferably the terminal is started in a separate window by clicking the application file [Bank_System.exe].

No specific commands or instructions need to be written to start the application. The user can immediately start using the application as soon as it runs. Users can exit the code any time through the exit button in the main menu

Program execution:

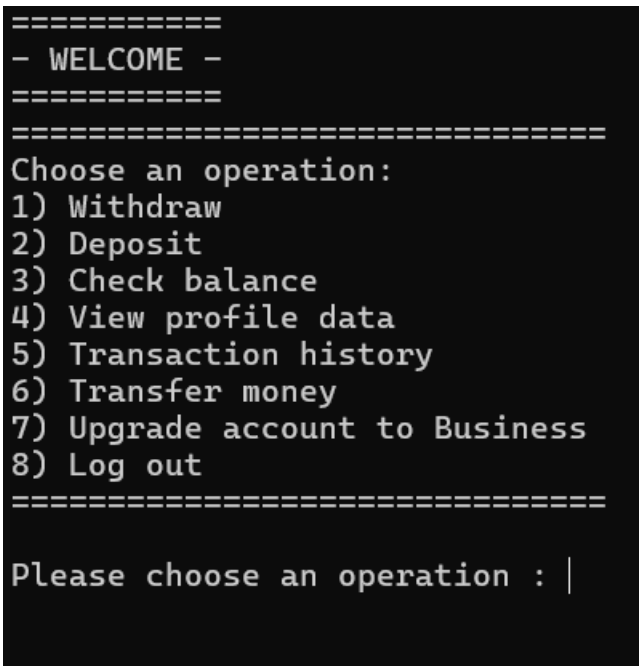
After running the file, the user is shown the main menu with 3 main options; create an account, sign in, or exit.



When creating an account, the user is taken to a new window where they can present their necessary data that is needed to create the

account such as names, national ID, phone number ...etc.

If a user wants to sign in instead then he enters the ID first and then is followed by a greeting message and then chooses which account to do operations on. A user can have a maximum of 2 accounts, a main one and a savings one. User can upgrade from a standard main account to a business main account at any time.
after successfully providing the PIN for the account the user is presented with options such as



After the user is done with the account a log out button must be pressed then it returns to the main menu.

Input and Output

Inputs: Names, IDs, PINs, balances, phone numbers, etc.

Outputs: Confirmation messages, balances, transaction history

File input/output: BANK.txt stores and loads account info

File format:

```
First name: Ahmed
Last name: El Sheikh
ID: 30405020201716
Account type: Standard
PIN: 2004
Address: Safa street
Phone number: 01011009345
Balance: 5000

========================================
```

Program structure:

Classes Used:

- **Account (Base class)**: Holds common information like balance, name, ID, PIN, etc. and has general methods like withdraw, deposit, view profile, getters, and methods controlling transactions.
- **Standard Account**: Inherits from Account. Adds limits for deposits, withdrawals, and transfers.
- **Business Account**: Inherits from Standard Account. Increase transaction limits.
- **Savings Account**: Inherits from Account. It cannot transfer money. It has a method to apply interest.
- **User**: Each user has one main account (Standard or Business) and/or one Savings account.

Functions used:

- PINcreation(): Creates a valid 4-digit PIN.
- save_file(): Saves all user accounts into a file.
- read_file(): Loads all user accounts from the file when program starts.
- after_colon(): Extracts data after a colon (used in file reading as it only extracts what is after the colon in the file as seen in the file format).

Advanced Features:

- **Inheritance** helps reuse and extend a class to subclasses.
- **Encapsulation** keeps sensitive data protected and all methods under the same class.
- **Polymorphism** allows different account types to behave differently with the same

functions (e.g., withdrawing behaves differently in each account type).

- **Maps** store users efficiently using their ID and eliminating the use of search algorithms as finding the user happens in O (1) time.
- **Try/Catch** blocks handle errors when reading numbers from the file.
- **File Handling** ensures user data is saved and loaded even after the program closes.
- **Enum** forces limited account types with limited string patterns to eliminate different strings such as (Standard, standard, StAnDaRd)
- **Functions from string library** isdigit(),.size(),
- **Vector** which carries a whole string containing each money transfer and receival

Testing and verification:

Invalid PIN

Adding larger balance limits to a standard account from the file.

Typing a spaced string such as full address

Upgrading account

Transferring money from a saving account

Improvements and extensions:

- Add time function to calculate interest automatically, edit transaction times and record age of the user.
- Create header files and divide the code into them.
- Add subclass (company)
- Add an admin panel to monitor accounts

Difficulties encountered:

- Using maps
- Inheritance and using polymorphism
- Using pointers

Conclusion:

C++ is generally an easier programming language than C. It is a good language to practice OOP which is very effective and secure. This project made me think like a user, a developer and a bank manager. I realized many methods and ideas throughout the project and thought of how to solve and execute them. Generally speaking, OOP is an important programming paradigm. Using maps may not be the most efficient way to store data; however, it's quick and easy to handle.

References:

My own lecture notes

C++ course/ object-oriented course from:

https://www.gammal.tech/

AI to generate ideas to add in the code